HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
School of Information and Communications Technology
Master's Programme in Data Science

BUSINESS INTELLIGENCE - GROUP PROJECT REPORT

Real-Time Payment Fraud Detection for an E-Commerce Platform

An End-to-End Analytics Project:
From Raw Data to a Monitored, Deployed Model

Course	Business Intelligence (IT5315E)
Instructor	Assoc. Prof. Bùi Quốc Trung
Domain	E-Commerce - Payment Risk & Trust
Report scope	Modules 1–8 (complete end-to-end lifecycle)

Team Members

Full name	Student ID
Lương Minh Dương	20251038M
Trần Lê Phương Thảo	20251186M
Hoàng Minh Hoàng	20252561M
Ngô Việt Anh	20252570M
Phạm Tuấn Kiệt	20252751M
Nguyễn Như Thái	20252270M

Hanoi, July 2026

Abstract

This report documents an end-to-end business-intelligence product that scores online payment transactions for fraud risk in real time. Acting as the risk-analytics function of an e-commerce platform, we answer a concrete business question: *which incoming transactions are likely to be fraudulent, and how should the platform balance blocking fraud against approving legitimate orders without unnecessary friction?*

We combine the public **PaySim** transaction dataset (6,362,620 records, fraud prevalence 0.129%) with a purpose-built **synthetic contextual layer** - identity, device, geolocation, and behavioural risk signals generated in Python with Faker plus custom business logic, each field documented in a full data dictionary. The pipeline proceeds through exploratory data analysis, conservative cleaning, leakage-aware feature engineering with explicit class-imbalance handling, and a comparative model-development stage that selects an operating threshold from a business cost function.

The central methodological contribution is a strict separation between *leaky* and *deployable* features. PaySim's post-transaction balance fields near-deterministically encode the label but are unknown at authorization time; models trained on them reach an artificial AUC-PR of ≈ 1.0. We exclude them from deployment and instead ship a model trained only on authorization-time signals. The deployable model - **XGBoost** on the 44-feature `realistic` group - achieves a held-out test **AUC-PR of 0.907**, **ROC-AUC of 0.999**, **recall of 98.1%**, and avoids **99.9%** of fraud loss while flagging only **2.4%** of transactions.

The model is then productionised as a complete analytics product: a **three-model ensemble** (Logistic Regression, Random Forest, XGBoost) served behind a FastAPI `/score` endpoint that returns each model's verdict plus a *max-risk* aggregate (`block > review > allow`); an eight-page Streamlit analyst application (executive overview, segment analytics, review queue with reason codes, model evaluation, cost/ROI, live feed, monitoring, and an API tester) deployable to a single free-tier port via an embedded API; and a monitoring layer that tracks per-feature and per-model Population-Stability-Index (PSI) drift, fires a retraining trigger at PSI ≥ 0.25, supports in-app retraining, and emits webhook alerts. This report documents the full lifecycle across all eight modules.

Keywords: fraud detection, class imbalance, XGBoost, cost-sensitive threshold, data leakage, PaySim, synthetic data, AUC-PR.

Contents

Chapter 1

Introduction and Business Understanding

1.1 Project context and business question

E-commerce platforms process thousands of transactions per minute. Even a fraud rate well under 5% translates into significant financial loss and eroded customer trust. This project asks the team to build, deploy, and monitor a classifier that flags high-risk transactions in real time, mirroring the full lifecycle of an analytics product in industry. The guiding business question is:

> *"Which incoming transactions are likely to be fraudulent, and how should the platform balance blocking fraud against approving legitimate orders without unnecessary friction?"*

The trade-off is inherently three-sided:

- **Missed fraud** (false negatives) causes direct financial loss, roughly proportional to the transaction amount.
- **False alarms** (false positives) create customer friction, potential churn, and manual-review labour.
- **Manual-review capacity** is finite, so every flag consumes a scarce resource.

A good policy therefore cannot be chosen by accuracy alone; it must be chosen against a *cost model* that prices these outcomes in a common currency. This principle drives our metric choice (AUC-PR and a cost-based objective rather than accuracy) and our operating-threshold selection (Section 6.4).

1.2 Key performance indicators (KPIs)

We define three KPIs that connect the model directly to business value.

KPI	Formula	Business meaning
Fraud rate (prevalence)	$N_{\text{fraud}}/N_{\text{txn}}$	Base rate the model must handle. PaySim: 8,213/6,362,620 = 0.129%.
False-positive rate	$FP/(FP+TN)$	Share of legitimate transactions wrongly flagged. Lower ⇒ less friction.
Financial loss avoided	$\frac{\sum \text{amount}(TP)}{\sum \text{amount}(\text{all fraud})}$	Share of fraudulent money intercepted before loss.

Table 1.1: Business KPIs used throughout the project.

The total exposed fraud amount in PaySim is **12,056,415,427.84** currency units; "loss avoided" is measured against this denominator.

Read as a set, the three KPIs describe different faces of the same decision and are designed to pull against one another. Prevalence (0.129%, a 774:1 imbalance) is the KPI a risk manager should internalise first, because it silently caps what the others can achieve: at this base rate a model that simply never flags anything is already 99.87% accurate, which is why accuracy is discarded outright, and it is also why precision will look structurally low - when genuine frauds are outnumbered 774 to one, even an excellent model must surface many legitimate transactions for each fraud it catches. The false-positive rate is the friction-and-capacity dial: because 6,354,407 legitimate transactions sit behind it, even a fraction-of-a-percent movement in FPR translates into tens of thousands of wrongly flagged customers and a proportional surge in review-queue volume, so this is the KPI that governs customer experience and operational load. Financial-loss-avoided is the north-star, and it is deliberately measured in money against the 12.06-billion-unit exposure rather than in fraud counts: it is superior to recall precisely because recall weights every fraud equally, whereas the business cares almost exclusively that the *largest* frauds are the ones intercepted - a model could catch most fraud cases and still haemorrhage money if it misses the few high-value ones. The tension is then explicit: raising loss-avoided requires lowering the decision threshold, which mechanically inflates the false-positive rate and the review load, and it is the cost model - not any single KPI - that decides where on this curve the platform should operate.

1.3 Cost model overview

Threshold selection (Module 5) minimises an *expected cost* that turns the three-sided trade-off into a single scalar:

$$\text{Cost} = \underbrace{c_{FN} \sum_{i \in FN} \text{amount}_i}_{\text{missed fraud}} + \underbrace{c_{FP} \cdot |FP|}_{\text{friction}} + \underbrace{c_{MR} \cdot |\text{flagged}|}_{\text{review labour}}. \tag{1.1}$$

The coefficients c_{FN}, c_{FP}, c_{MR} are business assumptions kept in `src/config.py` so that threshold selection and the report never drift apart. Their derivation, current values, and a sensitivity discussion are given in Section 6.4.

The three-sided trade-off matters precisely because its three arms are denominated in incommensurable units: missed fraud is a *variable* loss that scales with the transaction amount, a false alarm is a roughly *fixed* charge against the customer relationship (friction, churn, and the labour of unwinding a wrongly blocked order), and review capacity is a hard *operational* constraint on how many flags the team can physically clear in a day. No accuracy-style metric can arbitrate between them, because accuracy treats every transaction as one identical unit and every error as equivalent - it would score a missed multi-million fraud and a wrongly declined small purchase as the same mistake, which is exactly the judgement a risk function must never make. Collapsing the problem into the single expected-cost scalar of Equation 1.1 is therefore not a mathematical convenience but the correct decision lens: it forces the business to state its exchange rates explicitly (how many currency units of customer friction is one unit of fraud loss worth?), records those assumptions in one auditable place rather than leaving them implicit inside an analyst's threshold choice, and converts a subjective negotiation into an optimisation with a unique cost-minimising answer. Crucially, because the false-negative term is weighted by the amount at risk rather than by a flat per-case penalty, the objective is automatically amount-sensitive: it makes the resulting policy more willing to block a large transfer than a small payment - the exact risk-manager instinct that accuracy, and even F_1, are structurally unable to express.

The single most consequential quantity in the cost model is the ratio c_{FP}/c_{FN}, which fixes a break-even amount of 25 currency units. Its interpretation is stark: a suspected fraud is worth

blocking whenever the amount at stake exceeds 25 units, because a missed fraud costs the *full* amount while a false alarm costs a flat 25. The $c_{FN} = 1.0 \times$ amount assumption is not arbitrary - it reflects the mechanics of push-payment fraud, where funds are exfiltrated through TRANSFER and CASH_OUT and are effectively unrecoverable once the destination account is emptied, so the platform genuinely absorbs the whole amount. Since the median fraud runs to hundreds of thousands of units, essentially every fraud clears the 25-unit break-even by three to four orders of magnitude, and the economically rational policy is unambiguous: block aggressively and tolerate a large volume of false alarms in exchange for missing almost no material fraud. This is why the deployed operating point later reads as recall ≈ 0.98 against precision ≈ 0.17 - roughly five of every six flags are in fact legitimate. That low precision is a *designed* outcome of the cost arithmetic, not a modelling defect: each of the five false alarms costs on the order of 25 units (and only 3 to route through manual review), while the one genuine fraud among them saves hundreds of thousands, so the trade is overwhelmingly positive - the policy intercepts 99.9% of fraud value while flagging only about 2.4% of traffic. The reader should therefore expect the low precision reported in Module 5 and read a flag not as "this transaction is fraud" but as "the expected loss from *not* reviewing this exceeds the cost of reviewing it."

Insight - low precision is the policy, not a failure. With a break-even of just 25 currency units set against a median fraud of hundreds of thousands, the cost arithmetic *mandates* blocking widely. Precision near 0.17 is simply the price of intercepting 99.9% of a 12.06-billion-unit fraud exposure - and because each false alarm costs about 25 units against six-figure savings per catch, it is the cheapest price available. Fraud detection here is a cost-optimisation problem, and its optimum is deliberately high-recall by design.

1.4 Team and role distribution

The project follows the suggested five-role structure; with a six-person team the report/presentation duty is shared. Table 1.2 shows the proposed distribution (adjustable by the team).

Member	Role	Primary responsibilities
Lương Minh Dương	Data Engineer	Synthetic data generation, data dictionary, cleaning
Trần Lê Phương Thảo	Data Analyst	Exploratory data analysis, fraud-pattern visualization
Hoàng Minh Hoàng	ML Engineer (Modeling)	Feature engineering, classifier development
Ngô Việt Anh	ML Engineer (Evaluation)	Imbalance handling, threshold and cost-based evaluation
Phạm Tuấn Kiệt	MLOps Lead	Deployment (API + review queue), drift monitoring
Nguyễn Như Thái	MLOps / Reporting	Monitoring support, final report and presentation

Table 1.2: Proposed team roles (adapted from the project brief; to be confirmed by the team).

1.5 Repository and reproducibility

All code is authored by the team (no AutoML platforms). The pipeline is fully scripted and reproducible from a single fixed seed (`SEED=42`); Table 1.3 maps each module to its source. Heavy artefacts (raw CSV, processed parquet, model bundles) are regenerated locally rather than committed.

#	Module	Source files
1	Business understanding & data generation	`src/synth_context.py`, `src/build_dataset.py`, `docs/data_dictionary.md`
2	Exploratory data analysis	`src/eda.py`
3	Data cleaning	`src/cleaning.py`
4	Feature engineering	`src/features.py`, `src/imbalance.py`
5	Model development & ensemble	`src/train_validate.py`, `src/ensemble.py`, `src/infer.py`
6	Deployment	`api/main.py`, `app/*.py` (8-page app), `app/embedded_api.py`, `src/costs.py`, `src/reasons.py`
7	Monitoring	`monitoring/drift.py`, `app/monitoring_view.py`, `src/retrain.py`, `src/alerting.py`
8	Report & presentation	this document

Table 1.3: Module-to-file map. Steps 1–5 run together via `./train.sh full`.

Chapter 2

Data Sources and Synthetic Data Generation (Module 1)

2.1 Base dataset: PaySim

The required base is the Kaggle *Online Payments Fraud Detection Dataset* [2] (dataset id `rupakroy/ online-payments-fraud-detection-dataset`), a PaySim simulation of mobile money transactions. Key structural facts, confirmed on the full file:

- **6,362,620** transactions over **743** hourly steps (≈30 days).
- **8,213** frauds - a prevalence of **0.129%** (severe imbalance).
- Fraud occurs *only* in `TRANSFER` and `CASH_OUT` transactions.
- Eleven base columns: `step`, `type`, `amount`, `nameOrig`/`nameDest`, four balance fields, `isFraud` (target), and PaySim's own rule flag `isFlaggedFraud`.

PaySim provides transactions, balances, and labels but *no* e-commerce risk context - no customer tenure, device fingerprint, geolocation, address mismatch, or failed-payment history. The project brief therefore requires a synthetic contextual extension.

2.2 Why and how we generate synthetic context

We add a transparent synthetic layer with *overlapping* fraud/legitimate distributions, so that no synthetic field perfectly separates the classes. This keeps the task realistic (the synthetic signals *support* detection rather than trivially solving it). The generator (`src/synth_context.py`) is organised into three layers:

- **L1 - Identity** (display/context only): `customer_id`, `customer_name`, `email`, `billing_city`. A fixed pool of 200,000 Faker identities is used because the real `nameOrig` is almost always single-use.
- **L2 - Account risk**: `account_age_days` (lognormal), `high_risk_country` / `billing_country`, `is_disposable_email`.
- **L3 - Transaction / device / time risk**: `is_new_device`, `shipping_billing_mismatch`, `num_failed_payment_attempts` (Poisson), `ip_billing_distance_km` (Haversine from home to a simulated IP), `browser`, `device_os`, time features, and illustrative account velocity.

2.2.1 The reveal/noise softening mechanism

Rather than drawing a risk flag directly from the label (which would create perfect leakage), each fraud-conditioned field is generated through an *effective risk mask*:

$$\text{risky}_i = (\text{isFraud}_i \wedge U_i < r) \ \vee \ (\neg\text{isFraud}_i \wedge V_i < n), \tag{2.1}$$

with *reveal rate* $r = 0.55$ and *legit noise* $n = 0.04$ ($U_i, V_i \sim \text{Uniform}(0, 1)$). Only when risky_i is true does the field draw from its "fraud" distribution. Consequently many frauds look benign and some legitimate rows look risky - exactly the ambiguity a real detector faces. Table 2.1 lists the parameters.

Field	Legit	Fraud	Distribution / note
`account_age_days`	$(6.0, 0.9)$	$(5.0, 1.0)$	Lognormal (μ, σ), clipped 1–3650 days
`high_risk_country`	0.06	0.28	Bernoulli; high-risk = NG/RU/CN/ID
`is_disposable_email`	0.04	0.22	Bernoulli; disposable vs. common domain
`is_new_device`	0.15	0.50	Bernoulli
`shipping_billing_mismatch`	0.08	0.42	Bernoulli
`num_failed_payment_attempts`	$\lambda = 0.30$	$\lambda = 1.40$	Poisson
`ip_billing_distance_km`	$(0.2, 1.0)$	$(1.4, 1.1)$	Lognormal offset (deg) $\rightarrow$ Haversine km

Table 2.1: Synthetic generation parameters. Reveal rate $r = 0.55$, legit noise $n = 0.04$, $n_{\text{customers}} =$ 200,000, `SEED`=42.

2.3 Data dictionary

Every synthetically generated field is documented with column name, data type, unit, valid range, and generation logic, as required by the brief. Table 2.2 covers the base PaySim columns and Table 2.3 the synthetic layer.

Column	Type	Unit	Range	Description
`step`	int	hours	1..743	Simulation time; one step = one hour.
`type`	cat	–	5 types	`PAYMENT/TRANSFER/CASH_OUT/CASH_IN/DEBIT`; fraud only in TRANSFER & CASH_OUT.
`amount`	float	currency	≥ 0	Transaction amount; proxy for fraud loss.
`nameOrig`	string	–	C###	Origin customer id; almost always single-use.
`oldbalanceOrg`	float	currency	≥ 0	Origin balance before.
`newbalanceOrig`	float	currency	≥ 0	Origin balance after (*post-transaction*).
`nameDest`	string	–	C/M###	Destination account; reused accounts are mule candidates.
`oldbalanceDest`	float	currency	≥ 0	Destination balance before.
`newbalanceDest`	float	currency	≥ 0	Destination balance after (*post-transaction*).
`isFraud`	int	flag	{0,1}	**Target label**.
`isFlaggedFraud`	int	flag	{0,1}	PaySim's built-in rule flag for very large transfers.

Table 2.2: Data dictionary - base PaySim columns.

Table 2.3: Data dictionary - synthetic contextual layer (Module 1). Fields are grouped by risk layer; a field is *fraud-conditioned* (softened via the reveal/noise mask) wherever its logic lists distinct legit/fraud parameters.

Column	Type / unit	Range	Generation logic / business assumption
L1 - Identity (display / context only)			
`customer_id`	string	U0..U199999	Assigned from a fixed synthetic customer pool because real `nameOrig` is single-use.
`customer_name`	string	Faker name	Faker name for review-queue / demo display only; not a model feature.
`email`	string	handle@domain	Faker handle + common or disposable domain per `is_disposable_email`.
`billing_city`	string	Faker city	Faker city for analyst-facing context.

continued on next page

Table 2.3 continued

Column	Type / unit	Range	Generation logic / business assumption
L2 - Account risk			
`account_age_days`	int / days	1..3650	Lognormal days; legit $(6.0, 0.9)$, fraud $(5.0, 1.0)$, via reveal/noise mask.
`billing_country`	cat	12 ISO codes	High-risk = NG/RU/CN/ID; selected by `high_risk_country`.
`high_risk_country`	int / flag	{0,1}	Bernoulli; legit 0.06, fraud 0.28.
`is_disposable_email`	int / flag	{0,1}	Bernoulli; legit 0.04, fraud 0.22.
L3 - Transaction / device / time risk			
`is_new_device`	int / flag	{0,1}	Bernoulli; legit 0.15, fraud 0.50.
`shipping_billing_mismatch`	int / flag	{0,1}	Bernoulli; legit 0.08, fraud 0.42.
`num_failed_payment_attempts`	int / count	≥ 0	Poisson; legit λ=0.3, fraud λ=1.4.
`browser`	cat	6 values	Uniform draw from common browsers.
`device_os`	cat	5 values	Uniform draw from common OSes.
`device_id`	string	D###	Random device fingerprint id.
`ip_billing_distance_km`	float / km	≥ 0	Haversine home→IP; offset deg legit $(0.2, 1.0)$, fraud $(1.4, 1.1)$.
`hour_of_day`	int / hour	0..23	Derived: `step` mod 24.
`day_index`	int / day	0..30	Derived: `step` // 24.
`is_night`	int / flag	{0,1}	Derived: 1 when hour $\in$ 0..5.
`account_txn_total`	int / count	≥ 1	Total synthetic-customer transactions (illustrative).
`account_txn_index`	int / count	≥ 0	0-based transaction order per synthetic customer.
`time_since_last_hours`	int / hours	-1 or ≥ 0	Hours since previous synthetic-customer txn (-1 if first).
`txn_count_last_24h`	int / count	≥ 0	Prior synthetic-customer txns in trailing 24h.

2.4 Design notes and honest limitations

- Faker identity columns are analyst-facing context, *not* real PII, and are dropped before modelling.
- Because real `nameOrig` is single-use, synthetic per-customer velocity is *illustrative*; the genuinely useful account-level signal comes from repeated `nameDest` (mule) aggregation, which we build causally in Module 4.
- PaySim's post-transaction balance fields are extremely predictive. We flag this early (Section 3.6) and treat balance-reconciliation features as a separate "leaky" track that is *excluded* from the deployed model.

Chapter 3

Exploratory Data Analysis (Module 2)

EDA is computed by `src/eda.py`. Raw-sensitive sections (type, amount, balance, `nameDest` reuse) use the full 6.36M-row CSV to avoid sampling artefacts; some context sections use a stratified processed sample (≈954k rows) that preserves the natural fraud rate.

3.1 Class imbalance

The dataset is severely imbalanced: 6,354,407 legitimate vs. 8,213 fraudulent rows (Figure 3.1). This 774:1 ratio dictates every downstream choice - we prefer AUC-PR over accuracy, use class weights, and select the threshold from a cost function rather than the default 0.5.

Figure 3.1: Class imbalance (log scale): fraud is only 0.129% of transactions.

The 774:1 ratio is best read as a needle-in-a-haystack problem: for every fraudulent transaction the model must sift through roughly 774 legitimate ones. Its first consequence is that accuracy becomes actively misleading. A degenerate classifier that labels *every* transaction legitimate already scores 99.87% accuracy, yet it catches zero fraud and averts none of the 12.06 billion in exposure. A high accuracy figure here is therefore evidence of nothing - it merely restates the base rate. This is why we abandon accuracy for AUC-PR, whose no-skill baseline is the prevalence itself (≈ 0.00129) and which credits the model only for genuine precision-recall gains on the minority class.

The same rarity explains the wide gap between our headline ROC-AUC (0.999) and AUC-PR (0.907) for XGBoost: with 774 negatives per positive, even a minute false-positive rate floods the

far larger negative pool, so ROC-AUC saturates near 1.0 while precision stays modest (0.173). Imbalance also propagates into the operating point. It forces class weighting during training and a threshold drawn from the cost function rather than the naive 0.5, because at a 0.129% base rate the default cut-off would simply optimise for the negatives that dominate the loss. In short, the class ratio is not a preprocessing nuisance - it dictates the metric, the objective, and the recall-first posture of the entire pipeline.

The accuracy trap. At a 0.129% fraud rate, a model that predicts "always legitimate" is 99.87% accurate and 100% useless. What a high accuracy number measures here is rarity, not skill - which is exactly why this project is graded on AUC-PR, recall, and avoided loss instead of accuracy.

3.2 Fraud by transaction type

Fraud is structurally confined to two transaction types (Figure 3.2, Table 3.1): `TRANSFER` (0.77% fraud rate) and `CASH_OUT` (0.18%). `PAYMENT`, `CASH_IN`, and `DEBIT` contain *no* fraud. This is the single strongest structural prior in the data and motivates the `is_transfer` / `is_cash_out` indicator features.

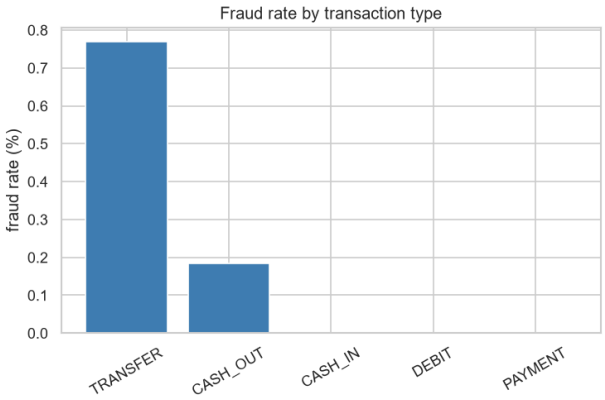


Figure 3.2: Fraud rate by transaction type.

Type	Frauds	Rate %
TRANSFER	4,097	0.769
CASH_OUT	4,116	0.184
CASH_IN	0	0.000
DEBIT	0	0.000
PAYMENT	0	0.000

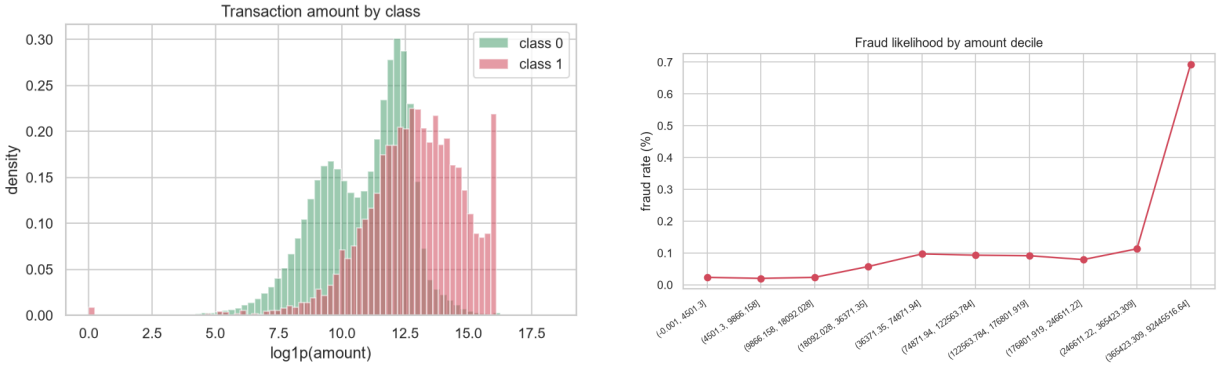
Table 3.1: Fraud counts and rates by type.

The confinement of all fraud to `TRANSFER` and `CASH_OUT` is not a sampling accident but a direct expression of how money is stolen. These are the only two channels that move value *out* of a victim's account and into the attacker's control: a `TRANSFER` relocates stolen funds to a mule account, and a `CASH_OUT` converts them to untraceable cash. The canonical scheme chains the two - transfer-in, then cash-out - which is precisely why the frauds split across both types (4,097 and 4,116). The three clean channels fail the attacker's economics: `PAYMENT` and `DEBIT` route funds to merchants for goods rather than returning liquid value, and `CASH_IN` *adds* money to an account, which no fraudster does to themselves. The zeros in Table 3.1 are therefore structural, not statistical noise.

Operationally this is the most valuable single finding in the EDA, because it hands the platform a free, near-perfect pre-filter: the roughly 60% of volume flowing through `PAYMENT`, `CASH_IN`, and `DEBIT` can be routed around the expensive scoring-and-review path with essentially no fraud risk in this data, concentrating scarce review capacity on the two channels that actually carry it. We encode the prior as `is_transfer`/`is_cash_out` indicators rather than a hard exclusion rule, for two reasons. First, it lets the model *combine* type with amount, time, and destination-reuse signals - a large night-time `TRANSFER` into a reused mule account is far riskier than the type alone implies. Second, it degrades gracefully if fraudsters ever migrate: a hard-coded rule would become a permanent blind spot the moment the attack pattern drifts, whereas a weighted feature simply loses influence.

3.3 Amount distribution and outliers

Transaction amounts are heavy-tailed: median ≈ 74,872, the 99.9th percentile ≈ 8.96M, and a maximum of 92,445,516. Fraudulent transactions skew towards larger amounts (Figure 3.3). Binning amount into deciles (Figure 3.3b) shows the fraud rate rising sharply in the top decile (0.69%) - an order of magnitude above the low deciles (≈0.02%). This motivates `log_amount` and confirms that amount is a genuine (if weak on its own) signal.



(a) Amount density by class ($\log 1p$). **(b)** Fraud rate by amount decile.

Figure 3.3: Transaction amount vs. fraud likelihood. Fraud concentrates in large amounts.

The rightward skew of fraud amounts reflects the attacker's optimisation, not chance. Each fraudulent attempt carries a roughly fixed cost - a compromised credential, a mule account, exposure to detection - so the rational fraudster maximises value extracted per attempt and pushes towards large transfers. This is exactly what the decile curve shows: the fraud rate climbs about 30-fold, from roughly 0.02% in the low deciles to 0.69% in the top decile. Yet the same figure carries the opposite warning - even in that top decile about 99.3% of transactions are legitimate, so amount *shifts the prior* without *separating* the classes. That is why amount is the strongest linear correlate of fraud and still weak in absolute terms ($r \approx 0.077$, with $|r| < 0.08$ for every feature): on its own it would drown genuine fraud in high-value legitimate commerce.

The decisive role of amount is therefore not as a standalone rule but as the bridge between the feature space and the cost model. It enters the pipeline twice. As an input it is log-transformed (`log1p`) to compress a tail spanning a 74,872 median to a 92.4M maximum, so a handful of extreme values cannot dominate the fit. As a misclassification weight it drives the objective directly, since a false negative costs the full amount ($c_{FN} = 1.0 \times$ amount) while a false positive costs a flat 25 and a manual review just 3. Because fraud *density* and economic *stakes* both peak in the large-amount region, data and objective agree: the model should be most aggressive precisely where the biggest losses hide. With a break-even of only 25 currency units against fraud amounts in the hundreds of thousands, blocking widely on a large-amount `TRANSFER` is the economically correct call even at low precision.

3.4 Temporal patterns

Fraud has a striking diurnal signature (Figure 3.4, left): the fraud rate peaks at roughly **22%** in the 04:00–05:00 window and is negligible during the day. The derived `is_night` flag captures this - night transactions carry a 1.77% fraud rate versus 0.10% otherwise. The per-day panel (right) shows a spike on the final simulation day; this is a low-volume simulation quirk (a handful of rows) rather than a real pattern, and we treat it as such.

The nocturnal peak - a fraud rate near 22% in the 04:00–05:00 window against a near-zero daytime baseline - reads as a behavioural signature of organised, automated attack rather than

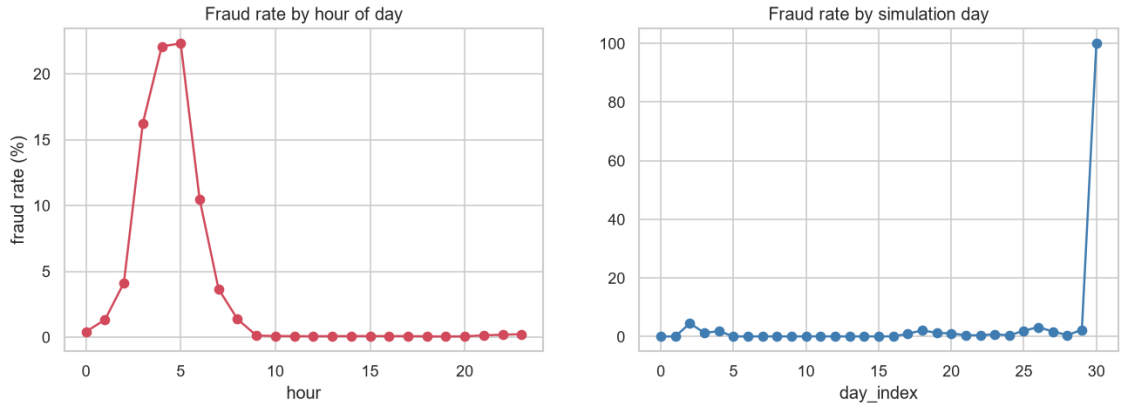


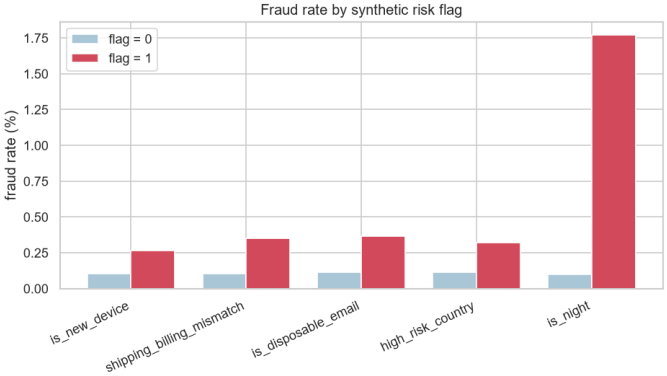
Figure 3.4: Fraud rate by hour of day (left) and by simulation day (right).

opportunistic misuse. Scripts fire in the small hours because monitoring is thinnest, fraud-operations staff are off-shift, and, critically, the genuine account-holder is asleep and unlikely to notice or reverse the transaction before the funds are cashed out. The magnitude of the effect sets time apart from every other signal: `is_night` lifts the fraud rate from 0.10% to 1.77% - about 17× - whereas the device, email, mismatch, and country flags manage only a 2.5–3.5× lift. That contrast is itself informative: the context flags are deliberately noisy proxies (engineered with `reveal_rate` 0.55 and `legit_noise` 0.04, so their distributions overlap), while time-of-day is a near-orthogonal behavioural tell the attacker cannot cheaply disguise without surrendering the low-monitoring advantage that motivated the night attack in the first place.

For deployment this makes time-of-day an almost ideal risk multiplier: it is free to compute, always available at authorization time, and tamper-resistant in a way that device fingerprints and email domains are not. The natural design lets `is_night` and the underlying hour modulate risk in interaction with type and amount rather than act alone. The temporal panel also delivers a discipline lesson: the day-30 spike to a 100% fraud rate is a low-volume simulation artefact - a handful of rows - and treating it as signal would teach the model a spurious rule. Recognising when *not* to trust a rate is as much a part of the analysis as the nocturnal pattern we keep.

3.5 Synthetic risk flags

The synthetic risk flags behave as designed (Figure 3.5, Table 3.2): each of the account/device/context flags roughly doubles-to-triples the fraud rate when set, while the temporal `is_night` flag lifts it far more - from 0.10% to 1.77% (≈17×). None is deterministic, however - the overlap engineered via the reveal/noise mechanism is visible and intended.



Signal	if 0 %	if 1 %
`is_new_device`	0.102	0.268
`shipping_billing_mismatch`	0.106	0.353
`is_disposable_email`	0.116	0.365
`high_risk_country`	0.114	0.320
`is_night`	0.100	1.773

Table 3.2: Fraud rate by flag value.

Figure 3.5: Fraud rate by synthetic risk flag.

Each account/device flag corresponds to a recognisable fraud tell, and the size of its lift tracks how directly the flag exposes adversarial intent. `shipping_billing_mismatch` carries the

largest identity-flag lift (0.106% to 0.353%, roughly 3.3×) because a ship-to address that diverges from the billing address is the operational signature of reshipping and drop-address schemes, where goods must physically reach someone other than the account holder. `is_disposable_email` (0.116% to 0.365%) reflects deliberate non-traceability: a throwaway inbox is cheap to mint and is favoured by synthetic-identity and bulk-fraud rings that expect the account to be burned. `is_new_device` (0.102% to 0.268%) is the classic account-takeover fingerprint - an unrecognised device authenticating against an existing account - while `high_risk_country` (0.114% to 0.320%) encodes the geographic concentration of organised fraud origination. The lone outlier is `is_night` (0.100% to 1.773%, ≈ 17×), a temporal rather than identity signal treated separately in the temporal-patterns section.

The analytically important point is not any single lift but its ceiling. Even the strongest identity flag leaves the flagged population overwhelmingly legitimate - a 0.353% post-flag fraud rate still means well over 99% of flagged transactions are genuine - so no flag on its own is remotely decisive; conditioning on it barely moves an individual transaction from "unlikely fraud" to "still unlikely fraud". This overlap is engineered on purpose (reveal rate $r = 0.55$, legit noise $n = 0.04$), but it is engineered to mirror reality: in production no single rule convicts, because a fraudster can defeat any one check in isolation. The discriminative signal lives in the *joint* configuration - a new device *and* a disposable email *and* an address mismatch together - which a rule-based scorecard cannot weigh but a multivariate model can. These marginals therefore justify a learned model over a threshold on any single flag, and they set up the non-linearity thesis developed below.

Numeric context fields tell the same story (Figure 3.6): fraudulent accounts are younger, sit farther from their billing IP, and have more failed payment attempts - but the distributions overlap heavily, so no single field is a silver bullet.

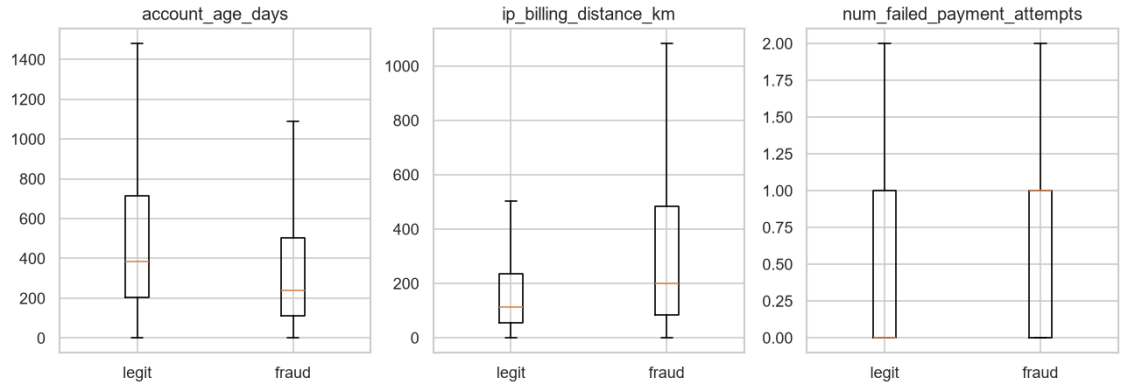


Figure 3.6: Numeric context by class: account age, IP–billing distance, failed attempts.

The numeric context fields sketch two distinct fraud archetypes rather than one. Fraudulent transactions come from accounts that are younger on average, sit farther from their billing IP, and carry more failed payment attempts; read as a pattern these point to (i) *account-takeover*, where an otherwise ordinary account is suddenly operated from an implausible distance on unfamiliar hardware, and (ii) *purpose-built fraud accounts* that are freshly minted and lightly aged. The elevated `ip_billing_distance_km` is the geographic-implausibility tell of a remote attacker using stolen credentials; the extra `num_failed_payment_attempts` is the residue of card-testing and credential-stuffing, where an adversary iterates over stolen instruments before one succeeds; and the lower `account_age_days` reflects both the youth of disposable fraud accounts and the recency skew of compromised ones.

Crucially, every one of these distributions overlaps its legitimate counterpart heavily, so none functions as a univariate classifier: the fraud and legitimate densities differ in their means but share almost all of their support. A hard threshold on account age or IP distance would either

miss most fraud or drown in false positives. This is the continuous-valued version of the same lesson the binary flags teach - each field is a weak, noisy nudge to the posterior, informative only in combination - and it is precisely why the modelling stage relies on an interaction-aware learner rather than any hand-set numeric cutoff.

3.6 Balance signature - an early leakage warning

PaySim's balance-reconciliation fields are *extraordinarily* predictive (Figure 3.7). The origin balance error and the "account drained" flag alone achieve directionless AUCs of 0.92 and 0.87 (Table 3.3). In particular, 97.6% of frauds drain the origin account (`orig_drained`) versus 23.8% of legitimate transactions.

Leakage warning. These signals are computed from the balance *after* the transaction settles, which is unknown at authorization time. A model that leans on them looks near-perfect offline but cannot be deployed. We therefore split features into a *leaky* reference track and a *deployable* authorization-time track (Chapter 5) and ship only the latter.

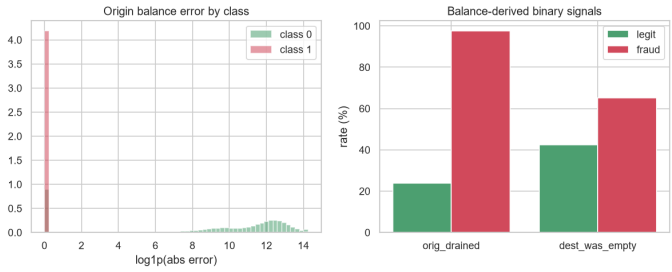


Figure 3.7: Origin balance error and balance-derived flags by class.

Feature	Dir.less AUC
`abs_errorBalanceOrig`	0.921
`orig_drained`	0.869
`dest_was_empty`	0.613
`abs_errorBalanceDest`	0.548

Table 3.3: Single-feature discriminative power of balance signals.

The balance signature is the report's most instructive *negative* result. A single feature, `abs_errorBalanceOrig`, reaches a directionless AUC of 0.921, and `orig_drained` fires for 97.6% of frauds against only 23.8% of legitimate transactions - numbers that would ordinarily be celebrated. Here they are a warning. The mechanism is that both features are reconciled from the balance *after* the transaction settles: draining the origin account is not so much a predictor of fraud as a near-definitional *consequence* of it, since exfiltration is the fraudster's objective. A feature that is an effect of the label, computed from post-event state, will always look near-perfect in an offline table - this is exactly the trap that target leakage sets.

The discipline this enforces is that offline AUC is not the deployment test - authorization-time availability is. At the instant the platform must decide, `newbalanceOrig` does not yet exist because the money has not moved. A model that leans on it cannot be served no matter how high its held-out score, and shipping it would yield a system that evaluates perfectly and catches nothing in production. This is why the balance fields are quarantined into a reference-only track and the deployable model is built exclusively from signals knowable *before* settlement - and why the honest 0.907 is headlined ahead of the leaky ≈ 1.0.

3.7 Destination reuse and the mule pattern

Although origin accounts are single-use, *destination* accounts are frequently reused: 2,722,362 unique destinations, of which 459,658 receive $\geq$2 transactions and one receives 113. Crucially, the fraud rate rises monotonically with reuse (Figure 3.8, Table 3.4): destinations seen 11+ times have a 1.69% fraud rate versus 0.12% for single-use destinations. This is the classic *mule-account* fan-in pattern and is the basis for our causal destination-history features (Module 4).

Figure 3.8: Destination reuse and fraud rate by reuse bucket.

Reuse	Dests	Fraud %
1	2,262,704	0.118
2–3	134,343	0.938
4–10	195,029	1.041
11+	130,286	1.692

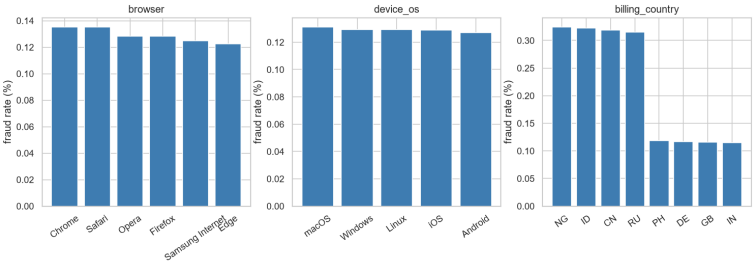
Table 3.4: Fraud-destination rate by reuse bucket.

The destination-reuse curve is the clearest causal signal in the data. Fraud rate climbs monotonically with how often a destination account has been seen - 0.118% at single use, 0.938% at 2–3, 1.041% at 4–10, and 1.692% at 11+, an escalation of roughly 14×. This is the fan-in geometry of money mules: a fraud ring funnels proceeds from many compromised origins into a small set of collector accounts, so a destination's accumulated inbound count is effectively a proxy for "this account is a collection point." The asymmetry with the origin side is the tell - origin accounts are essentially single-use (each victim or stolen credential is exercised once), so per-origin velocity carries no signal, whereas the destination is the reusable asset of the operation and therefore where the network signal concentrates.

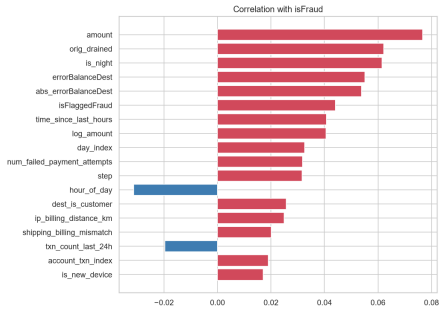
Analytically this reframes a transaction-level problem as a network-level one, and it dictates how the feature must be built. Because the fraud rate rises with reuse, the naive move - counting a destination's *total* appearances - would leak the future into the present, since a destination's final tally reflects transactions that have not yet occurred at authorization time. The deployable version is a *past-only* running aggregate: how many times, and from how many distinct senders, this destination has been seen *so far*. That converts a retrospective mule property into a causal, serve-time feature, and it is why destination history is engineered with cumulative sums computed before the chronological split rather than as a simple group-by count.

3.8 Channels and correlations

Browser and OS carry essentially no fraud signal (rates flat at ≈0.12–0.13%), as expected from uniform synthetic generation. Billing country does carry signal - the high-risk countries NG/ID/CN/RU top the chart at ≈0.32% (Figure 3.9a). Finally, linear correlations with the target (Figure 3.9b) are all individually weak ($|r| < 0.08$): `amount`, `orig_drained`, `is_night`, `errorBalanceDest`, and `isFlaggedFraud` lead. The weakness of *linear* correlations, contrasted with the strong tree-model results later, is itself evidence that fraud here is a non-linear, interaction-driven phenomenon.



(a) Fraud rate by browser, OS, and billing country.



(b) Top Pearson correlations with `isFraud`.

Figure 3.9: Channel context and linear correlations with the target.

The channel fields split cleanly into noise and signal, and both halves are informative about

data quality. Browser and device OS sit flat at the base rate (≈ 0.12–0.13%), which is exactly the correct result: the generator drew them uniformly and independently of the label, so any lift here would have signalled accidental leakage. Their flatness is a *negative control* that validates the synthetic hygiene of the contextual layer and licenses dropping them as model inputs. Billing country, by contrast, carries genuine geo-risk - the high-risk set (NG/ID/CN/RU) runs at ≈ 0.32% against ≈ 0.11% elsewhere - consistent with the real-world concentration of organised fraud in specific jurisdictions, and it is retained (frequency-encoded) rather than discarded.

The correlation panel delivers the report's central analytical bridge. Every linear correlation with the target is weak - all $|r| < 0.08$, with `amount` leading at just 0.077 - which, read naively, would suggest the features barely relate to fraud at all. The opposite is true, and the discrepancy is itself the finding: a weak *linear* association is not weak *signal*, it is signal that is not additive. Fraud here is conditional. A large amount is unremarkable for a seasoned account settling with a familiar destination but alarming for a young account transferring at 4am to a heavily-reused mule; the same feature value flips meaning depending on the others. Pearson correlation, which measures a single monotone main effect, is structurally blind to exactly this kind of interaction.

This is why the EDA points directly at a model class. If fraud were linearly separable a logistic regression would recover most of it and the correlations would be strong; the fact that they are not predicts that a linear model will underperform and that a learner able to represent conjunctions - recursive-partitioning trees - will not. The modelling stage should therefore be read as a test of this hypothesis rather than a routine bake-off: the weak marginals observed here are the reason gradient-boosted trees are the expected winner well before any model is trained.

EDA takeaways. (1) Extreme imbalance ⇒ AUC-PR + cost-based threshold. (2) Fraud lives only in TRANSFER/CASH_OUT, at night, in large amounts. (3) Balance fields are leaky ⇒ deployable-vs-reference split. (4) Destination reuse is a real, causal mule signal. (5) Signals are individually weak and overlapping ⇒ a non-linear model is needed.

Chapter 4

Data Cleaning (Module 3)

4.1 Philosophy: validate, document, preserve

PaySim is a clean simulation, so aggressive cleaning would destroy signal rather than add value. Our policy (`src/cleaning.py`) is deliberately conservative: *validate and document* everything, but *remove* only exact duplicates or negative amounts, and *preserve* the balance-reconciliation quirks because they are predictive (their leakage risk is handled at the feature stage, not by deletion).

4.2 Findings and decisions

- **Missing values:** none in any of the 33 input columns.
- **Duplicates:** 0 exact duplicates on the PaySim base columns.
- **Negative amounts:** 0. **Steps outside 1..743:** 0.
- **Zero-amount rows:** 16 found - kept and marked with a new `flag_zero_amount` column rather than dropped.
- **Category validation:** `type`, `browser`, `device_os`, `billing_country` all match their expected value sets exactly; no normalization needed.
- **Outliers:** amounts above the 99.9th percentile (6,363 rows) are kept but an optional `amount_capped` column (winsorised at p99.9) is added for robust modelling.
- **Balance quirks preserved:** 2.32M rows with zero destination balances before/after, and millions of reconciliation "errors", are retained as signal.

Table 4.1 gives the before/after summary. Net effect: two audit columns added, zero rows removed - a fully documented, reversible cleaning step.

Metric	Before	After	Change
Rows	6,362,620	6,362,620	0
Columns	33	35	+2
Fraud rows	8,213	8,213	0
Fraud rate	0.129%	0.129%	0
Missing cells	0	0	0
Duplicate base rows	0	0	0
Negative amounts	0	0	0
Zero-amount rows	16	16 (flagged)	0

Table 4.1: Cleaning before/after summary. Two columns added (`flag_zero_amount`, `amount_capped`); no rows removed.

Chapter 5

Feature Engineering and Imbalance Handling (Module 4)

5.1 The authorization-time principle

The overriding design rule is **causality**: a feature may use only information available at the moment the transaction is authorized. This rules out post-transaction balances and any aggregate that peeks at future rows. Getting this wrong is the most common way fraud models fail in production, so we enforce it structurally, not just by convention.

5.2 Chronological split (no test peeking)

We split *by time* on `step` (Table 5.1), not randomly, so training always precedes validation/test - a realistic "predict the future" setup that also surfaces temporal drift.

Split	Rows	Fraud rate	Step window
Train	4,453,834	0.0818%	1–323
Validation	954,393	0.0589%	323–378
Test	954,393	0.4200%	378–743

Table 5.1: Time-based 70/15/15 split. The higher test fraud rate reflects genuine temporal non-stationarity - a useful stress test.

5.3 Feature families

Four families of features are engineered (`src/features.py`):

1. **Base PaySim:** `amount`, `log_amount`, `amount_cents` (card-testing / round-number signal), pre-transaction balances, transfer/cash-out flags, and - in the reference track only - balance-reconciliation errors and drained/empty flags.
2. **Destination history (causal, past-only):** 16 features aggregating each `nameDest`'s *prior* behaviour - transaction count so far, amount sum/mean/std so far, unique senders so far (fan-in), per-type counts, recency (`time_since_dest_last_seen`), and anomaly ratios (`amount_to_dest_mean_ratio`, `amount_dest_zscore`). These are computed once on the full chronological frame *before* splitting via cumulative sums, so validation/test rows inherit real training-era history but never see the future.
3. **Synthetic / context:** the Module-1 risk signals plus derived time features and illustrative velocity.

4. **Categorical encodings:** fixed one-hot for `type`; train-frequency encoding for `browser`, `device_os`, `billing_country`. Raw IDs/display fields (`nameOrig`, `nameDest`, `customer_id`, `email`, ...) are dropped from the model matrix.

5.4 Feature groups: leaky vs. deployable

To quantify the cost of avoiding leakage, features are organised into five overlapping groups (Table 5.2). The two groups containing post-transaction balances are marked *leaky* and serve only as an upper-bound reference; the deployed model is chosen from the deployable groups.

Group	#feat.	Status	Contents
`base`	18	leaky	Full PaySim amount/balance signature (incl. post-txn balances).
`dest`	16	deployable	Past-only destination-account behaviour.
`synth`	21	deployable	Synthetic/contextual risk + encoded categoricals.
`realistic`	44	deployable **(deployed)**	Authorization-time base + destination history + context.
`all`	50	leaky	Everything (base + dest + synth + encodings).

Table 5.2: Feature groups. `realistic` is the deployable superset and drives the saved bundle; `base`/`all` include leaky balance fields and are reference-only.

5.5 Leakage-aware transformer

A single `FeatureTransformer` object encapsulates all state that must be *train-fitted*: the frequency-encoding maps, a median imputer, and a `StandardScaler` (for the linear model). It exposes two matrices - a `tree` matrix (raw, NaN-tolerant) for XGBoost/Random Forest and a `linear` matrix (imputed + scaled) for Logistic Regression. Validation/test data are only ever *transformed* with train-fitted state, eliminating train/serve skew. The transformer even refuses to run on raw subsets that lack precomputed destination history, turning the causality rule into an enforced invariant.

5.5.1 Automated leakage checks

Four assertions run on every build (Table 5.3); all pass. These guard the two most dangerous mistakes: a destination's first-ever transaction leaking a non-zero history, and raw identifiers slipping into the matrix.

Check	Status
First transaction to each destination has `dest_txn_count_so_far`= 0	PASS
`time_since_dest_last_seen`= −1 for unseen destinations	PASS
Raw display/ID columns absent from the feature matrix	PASS
Transformer rejects raw subsets lacking precomputed destination history	PASS

Table 5.3: Automated leakage checks (all pass).

5.6 Handling severe class imbalance

With a train fraud rate of 0.0818%, the negative:positive ratio is ≈1,221:1. We address this through:

- **Cost-sensitive learning (default):** `class_weight="balanced"` for Logistic Regression and Random Forest, and `scale_pos_weight` $\approx$ 1,221.57 for XGBoost. This re-weights the minority class without discarding or fabricating data.
- **Resampling (optional, train-fold only):** `src/imbalance.py` provides SMOTE and random undersampling, applied *strictly inside the training fold after transformation* - validation/test folds are never resampled, so evaluation stays honest.
- **Metric choice:** evaluation uses AUC-PR and the cost function, which remain informative under imbalance where accuracy and ROC-AUC can be misleadingly high.

Constant columns are dropped on train (0 in this build). The final matrices carry 50 tree features / 50 linear features, of which 16 are destination-history and 8 are encoded categoricals.

Chapter 6

Model Development and Evaluation (Module 5)

6.1 Methodology

Model development (`src/train_validate.py`) follows a disciplined protocol designed to prevent optimistic bias:

1. `prepare_feature_frame` runs *once* before splitting so causal destination history is consistent across folds.
2. Every (model × feature-group) pair is trained on `train`.
3. The **operating threshold** is chosen on `validation` by minimising the expected cost of Eq. (1.1).
4. The **best deployable configuration** is chosen on `validation` cost - never on test.
5. `test` is evaluated **exactly once** for the selected configuration.
6. Leaky groups (`base`, `all`) are *excluded from bundle selection*; they remain in the comparison table only as an upper-bound reference.

6.2 Candidate models

Three families required by the brief are compared (Table 6.1). XGBoost is a gradient-boosted tree ensemble expected to excel on the non-linear, interaction-heavy signal the EDA revealed; Random Forest is a bagged-tree baseline; Logistic Regression is a linear baseline that also stress-tests whether the problem is linearly separable (the EDA suggested not).

Model	Matrix	Key hyperparameters
Logistic Regression	linear	`max_iter=1000`, `class_weight=balanced`
Random Forest	tree	`n_estimators=120`, `min_samples_leaf=2`, `class_weight=balanced_subsample`
XGBoost	tree	`n_estimators=220`, `max_depth=5`, `learning_rate=0.08`, `subsample=0.9`, `colsample_bytree=0.9`, `scale_pos_weight`≈1221, `eval_metric=aucpr`

Table 6.1: Model families and hyperparameters.

6.3 Evaluation metrics for imbalanced data

We report metrics suited to a 0.13%-prevalence problem: **AUC-PR** (primary - the area under the precision–recall curve, which unlike ROC-AUC is sensitive to the minority class), **ROC-**

AUC, **precision / recall / F1** at the operating point, the **expected cost** of Eq. (1.1), and **loss avoided %** (share of fraudulent money intercepted).

6.4 Cost-based threshold selection

The threshold is not fixed at 0.5; it is swept over a grid and the value minimising validation expected cost is chosen. The three cost coefficients are derived rather than guessed (methodology in `docs/cost_model_worksheet.md`); the values currently in `src/config.py` are:

Coefficient	Value	Rationale
c_{FN} (missed fraud)	1.0× amount	Push-payment fraud has near-zero recovery; we lose ≈100% of the amount.
c_{FP} (false alarm)	25.0 / case	Lost margin + churn risk + support cost for a wrongly blocked customer.
c_{MR} (manual review)	3.0 / case	Fully-loaded analyst labour per reviewed transaction.

Table 6.2: Business-cost coefficients (assumptions; see the cost-model worksheet for the sourcing method).

6.4.1 Break-even and why the model blocks widely

Only the *ratio* of coefficients matters. The break-even amount - the fraud value at which blocking is worthwhile even at the risk of a false positive - is

$$\text{break-even} = \frac{c_{FP}}{c_{FN}} = \frac{25}{1} = 25 \text{ currency units.}$$

Because the median fraudulent amount in PaySim is enormous (hundreds of thousands), essentially *every* suspected fraud is worth blocking. The rational consequence is an operating point with **very high recall and modest precision**: the model deliberately casts a wide net because a missed large-value fraud costs far more than a review. This is a correct, intended outcome of the cost model, not a defect - and it is why we report loss avoided and flagged-rate alongside precision.

Recommended next step (sensitivity analysis). Because the coefficients are assumptions, the report should sweep $c_{FP} \in \{10, 25, 50, 100, 200\}$, re-select the threshold each time, and show that the operating point (recall, precision, flagged rate) is stable across a plausible range. A helper (`src/cost_sensitivity.py`) is proposed for Module 8.

6.5 Results

6.5.1 Deployable model comparison (`realistic` group)

Table 6.3 compares the three models on the deployable `realistic` group. **XGBoost** is the clear winner on every axis that matters: the highest test AUC-PR (0.907) and F1 (0.294), while still catching 98.1% of fraud and avoiding 99.9% of loss. Logistic Regression confirms the EDA's hint - a linear model cannot separate this data (test AUC-PR 0.587) and compensates with an untenable flag rate. Random Forest is competitive but trails XGBoost.

The three-way model gap is best read as a direct measurement of *where* the fraud signal lives. The EDA established that every linear correlation with the label is weak ($|r| < 0.08$) while risk concentrates in *conjunctions* - a large amount *and* at night *and* to a frequently reused destination. Logistic Regression is additive by construction: it can weight each signal but cannot represent the AND that makes them jointly damning, so on the deployable features it tops out

Model (realistic)	AUC-PR	ROC-AUC	Precision	Recall	F1	Loss avd.
XGBoost	**0.907**	0.999	0.173	0.981	**0.294**	99.91%
Random Forest	0.812	0.974	0.084	0.953	0.155	99.60%
Logistic Regression	0.587	0.990	0.025	0.997	0.049	99.98%

Table 6.3: Held-out *test* metrics on the deployable `realistic` group (threshold selected on validation). XGBoost is selected for deployment.

at 0.587 AUC-PR. The 0.32-point gap up to XGBoost's 0.907 is precisely the portion of the signal that is interaction-driven and therefore structurally invisible to a linear model. Recursive tree splits encode exactly those conjunctions; Random Forest does too, which is why it reaches 0.812, but XGBoost's sequential residual-fitting extracts more discriminative structure from the extreme minority class than Random Forest's independent bagging, accounting for the remaining ≈0.10-point margin.

Logistic Regression's headline recall of ≈1.0 is a trap, not a triumph. Unable to isolate fraud by interaction, the only way it can cover the positives is to slide its threshold down until it flags almost everything - which is exactly what a precision of 0.025 (roughly one true fraud per forty flags) reveals. That is an operationally dead policy: the review queue would drown. XGBoost reaches comparable recall (0.981) while holding precision an order of magnitude higher, because it can be confident about *which* transactions are risky rather than merely that some are. Two further properties seal the choice. First, `scale_pos_weight`≈1,221 lets the booster take the 774:1 imbalance seriously without discarding or synthesising data. Second, its native tolerance of missing values matters operationally: a live authorization request has no destination history (those features are simply absent at request time), and a tree routes around a missing split where the scaled linear model would need imputation that distorts the very tail in which the fraud lives.

6.5.2 Feature-group ablation (XGBoost)

Holding the model fixed at XGBoost and varying the feature group (Table 6.4) makes the leakage story concrete. The leaky groups (`base`, `all`) reach an artificial test AUC-PR of ≈1.0 - driven entirely by the post-transaction balance fields. Among deployable groups, neither destination history alone (`dest`) nor context alone (`synth`) is sufficient; only their combination in `realistic` yields strong, honest performance (0.907). This is the core empirical justification for the deployed feature set.

Group	Leaky?	#feat.	Test AUC-PR	Test recall	Loss avd.
`base`	yes	18	1.000†	0.9995	99.99%
`all`	yes	50	1.000†	0.9995	99.99%
`realistic`	no	44	**0.907**	0.981	99.91%
`synth`	no	21	0.243	0.961	96.21%
`dest`	no	16	0.009	0.998	99.77%

Table 6.4: XGBoost across feature groups. †Leaky groups are an upper-bound reference only - *not* deployable. The deployable optimum is `realistic`.

The ablation makes the cost of honesty measurable rather than rhetorical. Holding the model fixed as XGBoost and swapping only the feature group, test AUC-PR falls from the leaky groups' ≈1.0 to the deployable `realistic` group's 0.907 - a *leakage tax* of roughly nine AUC-PR points. That gap is not a modelling weakness; it is the exact quantity of apparent skill the leaky model was borrowing from information the platform does not possess at the moment it must decide. The two numbers are also not on equal footing. A model that scores 1.0 on post-settlement balance fields cannot be computed at authorization time at all, so its *deployable* discriminative power is

effectively zero: it never runs. Measured against what can actually be shipped, the 0.907 model is therefore not nine points worse than the leaky benchmark - it is the only model that exists.

Crucially, the tax is paid almost entirely in ranking sharpness and precision, not in the business objective. Even stripped of every balance feature, the deployable model still intercepts 99.91% of fraudulent money (Table 6.5) - statistically indistinguishable from the leaky upper bound's loss-avoided. What the platform forfeits by refusing leakage is thus not caught fraud but review efficiency: the honest model routes a few more borderline cases to human review to reach nearly the same interception. This reframes the leaky-versus-deployable split from a painful accuracy sacrifice into a cheap insurance premium - a handful of AUC-PR points and some extra queue load - bought in exchange for a model that does not silently collapse the instant it faces a live transaction whose post-transaction balances are, by definition, still unknown.

The leakage tax. The distance from the leaky ≈1.0 AUC-PR to the deployable 0.907 is the price of using *only* information available at authorization time - about nine AUC-PR points. It is a bargain: the honest model still avoids 99.91% of fraud loss. An offline score of 1.0 that cannot be computed when the decision is actually made is worth exactly nothing; a 0.907 that runs at authorization time is worth everything.

6.5.3 Selected model - the deployed configuration

The saved bundle (`models/fraud_model.joblib`) is **XGBoost on `realistic` (44 features)** with a validation-selected threshold of **0.241**. Its full profile is in Table 6.5. The validation→test AUC-PR change (0.806→0.907) is favourable and consistent with the higher, more separable fraud regime in the later test window; ROC-AUC is essentially identical across folds (0.999), indicating stable ranking.

Metric	Validation	Test
AUC-PR	0.806	0.907
ROC-AUC	0.9990	0.9987
Operating threshold	0.241 (selected on validation)	
Precision	0.029	0.173
Recall	0.989	0.981
F1	0.056	0.294
Flagged rate	2.03%	2.38%
Loss avoided	99.98%	99.91%

Table 6.5: Deployed model: XGBoost / `realistic` / 44 features / threshold 0.241.

The validation→test movement rewards a careful reading, because a model that scores *higher* on the future (0.907) than on validation (0.806) can look suspicious at first glance. It is not overfitting; it is a property of the metric under a shifting base rate. AUC-PR's no-skill floor equals the positive prevalence, and the test window carries a materially higher fraud rate than the validation window (Table 5.1). A denser positive class mechanically lifts AUC-PR even when the underlying ranking is unchanged - and the near-identical ROC-AUC across folds (0.999), which is prevalence-invariant, confirms exactly that: the model's ability to *order* transactions by risk did not degrade going forward. The AUC-PR gain reflects a later, more-separable fraud regime, not a better-fit model.

This is why the chronological, predict-the-future split is the only defensible evaluation here. A random split would let the model train on transactions from the same hours - even the same reused destination accounts - that it is later scored on, leaking future information backwards and inflating every metric. Cutting on `step` forces training to precede scoring, so the destination-history features carry only genuine past behaviour and the reported numbers answer the question

the business actually asks: given everything known up to now, how well will the model do on transactions it has never seen? The favourable val→test move is then reassuring in the strongest possible sense - performance held up under real temporal non-stationarity rather than under a shuffled fantasy that quietly rehearses the answer.

6.5.4 Interpreting the operating point

At the chosen threshold on the test window the model flags 2.38% of all transactions, catches 98.1% of frauds, and intercepts 99.9% of the fraudulent money. Precision is 17% by design (Section 6.4): roughly five of every six flags are false alarms sent to review, which the cost model accepts because the alternative - missing a six-figure fraud - is far costlier. Two thresholds are exposed downstream (Module 6): the review threshold (0.241) and a high-confidence auto-block threshold, enabling an *allow / review / block* policy rather than a binary decision.

The 17% precision at threshold 0.241 is the single number most likely to be misread as failure, so it is worth deriving why it is optimal. Treat each decision as an expected cost. Allowing a transaction risks $p \times$ amount (missed-fraud cost $c_{FN} = 1.0\times$amount); flagging it costs roughly c_{FP} plus a review - about 25–28 currency units almost regardless of p. Blocking therefore pays off whenever $p \times \text{amount} > 25$, the break-even of the coefficients in Eq. (1.1). Because the median fraud runs to hundreds of thousands, this inequality holds at fraud probabilities of a fraction of a percent. The cost structure literally *instructs* the model to flag on the faintest suspicion attached to a large-value transaction, and low precision is the arithmetically correct response, not a defect. Reporting loss-avoided (99.91%) and flagged-rate (2.38%) beside precision is what keeps the picture honest: the model catches 98.1% of fraud while touching only 2.38% of traffic.

What stops the threshold from collapsing to zero - the degenerate policy Logistic Regression fell into - is the review-labour term c_{MR}. Every flag consumes a scarce analyst, so the cost function charges for the queue itself; the 2.38% flagged rate is the point at which the marginal review cost of one more flag just balances the marginal fraud it expects to catch. This ties the statistical operating point directly to an operational constraint: a precision of 17% is affordable only because the queue can absorb roughly one flagged transaction in forty-two, and it is why the downstream policy prioritises the queue by *expected loss* (risk × amount). When capacity binds, scarce analyst time should chase the largest exposures first rather than the highest scores - precision is a review-capacity problem, not a modelling ceiling.

Honesty note. We deliberately do *not* headline the AUC-PR≈1.0 achievable with balance features. Those numbers are real but non-deployable; reporting them as the result would be misleading. The deployable 0.907 is the number the business can actually rely on at authorization time.

6.6 From a single model to a deployed three-model ensemble

Rather than ship one classifier, the serving layer packages *all three* models trained on the deployable `realistic` group into a single bundle, `models/fraud_ensemble.joblib` (built by `src/train_validate.py`, consumed through `src/ensemble.py`). The bundle stores one shared `FeatureTransformer` plus, per model, its fitted estimator, feature matrix (`tree`/`linear`), feature list, and *own* cost-tuned decision threshold. The single best model (`fraud_model.joblib`) is retained separately for batch scoring (`src/infer.py`).

At scoring time every model votes independently and the verdicts are combined by a **max-risk** rule - the most severe decision wins (`block > review > allow`) - with per-model thresholds

and a shared high-confidence block floor of **0.9** (Eq. 6.1):

$$\text{decide}(p,\tau) = \begin{cases} \text{block} & p \geq \max(0.9,\ \tau) \\ \text{review} & \tau \leq p < \max(0.9,\ \tau) \\ \text{allow} & p < \tau \end{cases} \qquad \text{aggregate} = \max_{\text{severity}} \{\text{decide}(p_m, \tau_m)\}_m. \quad (6.1)$$

This design buys two things over a single model: (i) a max-risk *union* raises recall (any model can escalate a suspicious case), matching the cost model's preference for catching fraud; and (ii) **graceful degradation** - if one model errors at request time, its entry is dropped, the aggregate is computed from the survivors, and the response is flagged `degraded` rather than failing. The same `src/ensemble.py` scoring path backs both the API (single record, via `score_record`) and the Streamlit batch (via `score_batch`), so a transaction gets an identical verdict either way - train/serve parity by construction.

The max-risk aggregation is more subtly engineered than "take the most alarmed model." Read against the cost model, it applies a *union* at the cheap decision boundary and reserves an intersection-like bar for the expensive one. Because any single model can escalate a case to *review*, the ensemble's effective recall is at least that of its most sensitive member - exactly the bias the cost function wants, since a missed fraud is the costly error and a needless review is cheap. But escalation to *block*, the expensive customer-facing action, is gated behind the shared 0.9 confidence floor of Eq. (6.1), so a model can force hard friction only when it is genuinely certain. The union therefore raises recall where being wrong is cheap (an extra review) while the high floor protects precision where being wrong is costly (a wrongful block). This is precisely why bundling the weak Logistic Regression alongside the stronger trees is defensible: it can only ever add reviews, never force a block on its own.

The second benefit is availability under partial failure - a property a real-time authorization path cannot do without. A model that errors or times out must not be allowed to take the whole decision down, because failing open would wave fraud through while failing closed would block legitimate customers wholesale. Dropping the failed model, recomputing the aggregate from the survivors, and marking the response `degraded` converts a hard outage into a graceful, still-actionable verdict that carries its own audit trail. Combined with train/serve parity - the same `ensemble.py` path scores both the offline batch and the single live request - the ensemble is engineered less for a marginal metric gain than for the operational robustness a payment system actually requires at request time.

Reproducibility note. All results in this chapter are the **full-data (6.36M-row)** build, and the deployed app (Chapter 7) scores on that same build. The trained ensemble bundle (≈60 MB) is committed to git so the app works immediately after clone, but the processed feature dataset (≈526 MB on full PaySim) exceeds GitHub's 100 MB limit and is regenerated locally by the data pipeline; a committed ≈26 MB stratified sample (`context_sample.parquet`) lets a fresh clone run the app in *sample mode* without the full rebuild.

Chapter 7

Deployment (Module 6)

The deployable model of Chapter 6 is packaged into a working analytics product: a real-time scoring API and an eight-page analyst application, engineered to run on a single free-tier cloud port. All money figures across the API, the app, and this report come from one shared cost module (`src/costs.py`), so the three can never disagree on what a decision is worth.

7.1 Real-time scoring API

A FastAPI service (`api/main.py`) loads the ensemble bundle at start-up and exposes two endpoints. `GET /` is a health check listing the loaded models and the aggregate rule. `POST /score` accepts a transaction payload and, following the flow in Section 6.6, enriches the single record (destination-history features disabled because a live request has no prior context), scores it with all three models, and returns each model's probability and *allow / review / block* decision plus the max-risk aggregate verdict. A per-model failure is isolated: that model returns an `error` entry, the aggregate falls back to the survivors, and the response is marked `degraded`.

```
POST /score   ->   HTTP 200
{
  "models": {
    "logreg": {"model_name": "Logistic Regression",
               "fraud_probability": 0.63, "decision": "review",
               "review_threshold": 0.52, "block_threshold": 0.90},
    "rf":     {"model_name": "Random Forest",
               "fraud_probability": 0.95, "decision": "block", ...},
    "xgb":    {"model_name": "XGBoost",
               "fraud_probability": 0.88, "decision": "review", ...}
  },
  "aggregate": {"decision": "block", "rule": "max-risk"}
}
```

Figure 7.1: Example `POST /score` response: three independent verdicts plus the max-risk aggregate. Here Random Forest escalates to *block*, so the aggregate blocks.

7.2 Transparent reason codes

For every flagged transaction the review queue shows a short, human-readable list of *why* it was flagged (`src/reasons.py`). We deliberately use rule-based reason codes - e.g. "2 failed payment attempts", "high-risk country", "IP 1,300 km from billing", "account fully drained", "shipping $\neq$ billing", "new account (12d)" - rather than a black-box attribution such as SHAP: for a fraud analyst, and for a regulator, a concrete list of the risk signals actually present is more actionable and more defensible than a bar chart of feature weights. The reason codes *explain*; the ensemble score still *decides*.

7.3 Shared cost / ROI model

`src/costs.py` turns any decision vector plus ground-truth labels and amounts into a full money breakdown using the Section 6.4 coefficients: fraud exposure, caught vs. missed amount, false-positive cost, review labour, and the resulting *net savings* versus two baselines (do-nothing and review-everything), plus an ROI figure (savings per unit of operating spend). The Cost & ROI page lets an analyst edit the cost matrix live and see the cost-optimal threshold move - making the fraud/friction trade-off tangible in currency rather than abstract metrics.

7.4 Eight-page analyst application

A multi-page Streamlit application (`app/streamlit_app.py` via `st.navigation`) presents the product as a capstone narrative - pitch → problem → product → rigour → value → real-time → operations → serving (Table 7.1). Every analytical page carries a data-provenance badge, and model metrics are always reported on the held-out temporal test split.

Page	What it shows	Data
Overview	Executive KPIs: money saved, fraud loss avoided, recall/precision, model health	test
Segment Analytics	Where fraud concentrates: by type, hour, amount + risk-factor lift	full population
Review Queue	Triage with per-transaction reason codes; 3 models + max-risk verdict	2k sample
Model Evaluation	AUC-PR / ROC / PR curves, confusion matrix, cumulative-gains / lift	test
Cost & ROI	Prices the trade-off in money terms; per-model vs. ensemble; editable cost matrix	test
Live Feed	Simulated real-time scoring stream	simulated
Monitoring	PSI drift detection → retraining + alerting (Chapter 8)	simulated
API Tester	Score a hand-built transaction through the live `/score` endpoint	live API

Table 7.1: The eight pages of the Streamlit analyst application.

7.5 Single-port cloud deployment

Free hosts such as Streamlit Community Cloud and Hugging Face Spaces expose only one public port. To keep the live API Tester working without a second port, `app/embedded_api.py` launches the FastAPI service in a background thread on `127.0.0.1:8000` inside the container; the app calls it *server-side* over localhost, so the browser only ever talks to Streamlit (and if a local `uvicorn` is already running, the open port is detected and reused). Deployment then reduces to: commit the runtime artefacts (the ≈60 MB ensemble bundle and the preview CSV, kept in git via `.gitignore` exceptions; the ≈526 MB processed dataset exceeds GitHub's limit and is rebuilt locally, with a committed ≈26 MB stratified sample as a fresh-clone fallback), point Streamlit Cloud at `app/streamlit_app.py`, and let `requirements.txt` and `packages.txt` (`libgomp1` for XGBoost) install automatically.

Remaining for submission. Publish the app to a free tier and paste the resulting **live link** here (a graded deliverable), together with one or two screenshots of the review queue and monitoring pages.

Chapter 8

Monitoring (Module 7)

A deployed fraud model decays as fraud tactics and customer behaviour shift. Module 7 closes the loop with a monitoring layer that detects distribution drift, decides when to retrain, performs the retraining, and raises an alert - all surfaced on the Monitoring page of the analyst app.

8.1 Drift detection with the Population Stability Index

`monitoring/drift.py` quantifies drift with the **Population Stability Index (PSI)**, comparing a reference window (earlier half of the stream, by `day_index`) to a current window (later half):

$$\mathrm{PSI} = \sum_b (c_b - r_b)\ \ln\frac{c_b}{r_b}, \tag{8.1}$$

where r_b, c_b are the reference/current fractions in bin b (reference-quantile bins). PSI is transparent and dependency-free. We use the industry-standard bands PSI < 0.10 (stable), 0.10–0.25 (moderate - monitor), and > 0.25 (significant). Critically, PSI is computed both on the **input features** (`amount`, `account_age_days`, `ip_billing_distance_km`, failed attempts, new-device, hour) *and* on each model's **predicted-score distribution** (`PREDICTION_SCORE_logreg/_rf/_xgb`) - so both data drift and per-model concept drift are visible.

8.2 Retraining trigger and self-validation

A **retraining trigger** fires when any monitored PSI reaches **0.25**. The module validates its own detector: alongside the natural earlier-vs-later split it constructs a *simulated fraud campaign* (`inject_drift`: amounts ×1.25–1.6, IP distance ×1.8, extra failed attempts) and confirms the trigger stays quiet on the natural split but fires on the injected one (Table 8.1). This demonstrates the monitor catches a real shift without false-alarming on benign temporal variation.

Signal	**Natural split**		**Simulated campaign**	
	PSI	band	PSI	band
`amount`	low	stable	high	**SIGNIFICANT**
`ip_billing_distance_km`	low	stable	high	**SIGNIFICANT**
`num_failed_payment_attempts`	low	stable	elevated	moderate/SIG.
`PREDICTION_SCORE_*` (per model)	low	stable	elevated	moderate/SIG.

Table 8.1: Self-validating drift check: the natural split stays *stable* while the injected fraud campaign pushes feature and per-model prediction PSI past the 0.25 retraining trigger.

8.3 In-app retraining and alerting

When the trigger fires, two operational responses are available:

- **Retraining** (`src/retrain.py`): all three ensemble models are refit on a freshly generated, current-distribution labelled sample, *reusing* the deployed transformer and each model's hyperparameters, and each decision threshold is re-selected by validation expected cost. This is a fast in-memory adaptation (seconds) for the live demo; the on-disk bundle is left untouched, so the operation is safe to trigger from the dashboard.
- **Alerting** (`src/alerting.py`): a Slack/Discord-compatible webhook payload and a downloadable Markdown incident report are generated, listing the signals that breached the threshold and their PSI values. Delivery uses only the standard library, never raises, and sends nothing unless a webhook URL is configured (kept out of git via `.streamlit/secrets.toml`).

8.4 Monitoring dashboard

The Monitoring page (`app/monitoring_view.py`) has two tabs. **Reports** renders the committed `drift_report.md` and embeds the optional Evidently (`DataDriftPreset`) HTML report. **Live** recomputes drift on demand: a "simulate fraud campaign" toggle applies `inject_drift`, the PSI table (features plus per-model prediction rows) is shown colour-banded, overlaid reference-vs-current histograms explain each PSI value, and a retrain banner with the alert action appears when the trigger fires.

Remaining for submission. Capture the rendered Evidently dashboard and one Monitoring-page screenshot for the appendix, and (optionally) add rolling-window precision/recall as labels arrive, noting the label-delay caveat inherent to fraud feedback.

Chapter 9

Risk Policy, Limitations, and Future Work (Module 8)

Modules 1–7 translate directly into an operating policy for the platform's risk team. Each recommendation maps a signal to a concrete action - auto-block, manual review, or allow - realised by the ensemble's max-risk thresholds and made auditable by the review queue's reason codes.

9.1 Risk-policy recommendations

- **Auto-block** only very-high-confidence cases (any model $\geq$ the 0.9 block floor), reserving hard friction for the clearest fraud.
- **Route to manual review** the mid-confidence band (between each model's cost-tuned review threshold and the block floor) - where the cost model already places most flags - and show the analyst the reason codes so the decision is explainable.
- **Prioritise** the review queue by *expected loss* (risk score $\times$ amount) so scarce analyst capacity chases the largest exposures first.
- **Apply cheap structural pre-filters:** only `TRANSFER`/`CASH_OUT` can be fraud; a night-time, large-amount transfer to a frequently-reused destination warrants extra scrutiny even at a moderate score.
- **Escalate on drift:** when the Module-7 PSI trigger fires, retrain and alert before performance visibly degrades.

Which signals auto-block vs. review? (the brief's core M8 question.) *Auto-block* is reserved for score-driven certainty (block floor) - no single raw signal auto-blocks, because every synthetic signal overlaps between classes. *Manual review* is triggered by the mid-band score *and* is where high-severity reason codes (multiple failed attempts, high-risk country, drained account, large night-time transfer to a reused mule destination) concentrate. *Allow* covers the low-score majority. This keeps auto-block precise and routes ambiguity to humans - matching the cost model's high-recall, review-heavy operating point.

The brief's central policy question - which signals auto-block and which merely flag for review - is settled by the data's structure rather than by preference. Every signal available at authorization time overlaps heavily between fraud and legitimate traffic *by construction*: the synthetic flags were generated with a reveal rate of 0.55 and a legit-noise of 0.04 precisely so that no clean separator can exist. The consequence is quantitative and decisive. The sharpest single flag, `is_night`, lifts the fraud rate roughly seventeenfold (0.10% $\rightarrow$ 1.77%) - yet even conditioned on a night-time transaction, more than 98% of transactions are still legitimate. The next-strongest flags (disposable email, shipping/billing mismatch, high-risk country, new device) each move

the rate only from about 0.11% to 0.27–0.37%, a 2.5–3.5× lift that leaves the posterior fraud probability well under one percent. Auto-blocking on any one of these would therefore reject overwhelmingly good traffic; the reason-code lifts are genuine, but individually far too weak to justify hard friction.

Certainty sufficient to auto-block emerges only after the model integrates many weak, overlapping signals into a calibrated posterior. The correlation analysis reports the same fact from the opposite side: no signal correlates with the target above $|r| = 0.08$, yet the tree ensemble reaches a test AUC-PR of 0.907 - fraud here is a non-linear conjunction of individually unconvincing signals, not any single axis. The policy tiers follow directly. Auto-block is gated on *score* certainty (any model $\geq$ the 0.9 block floor), a threshold on the model's integrated judgement and never on a raw input. The mid-band between each model's cost-tuned review threshold and the block floor - where the score is genuinely uncertain - is routed to a human, and it is exactly here that high-severity reason codes (multiple failed attempts, a large night-time transfer to a reused destination, a drained account) earn their place: they do not decide, they corroborate and explain, turning an opaque probability into an auditable narrative for the analyst and the regulator. The score decides; the reason codes justify.

At the chosen operating point the model flags 2.38% of a 6.36-million-transaction stream at a precision of 0.173 - roughly five false alarms for every genuine fraud. Human review capacity, not the classifier, is the binding constraint on how many of those flags can actually be worked, which recasts the queue as a scheduling problem: given a fixed number of analyst-hours, which flags earn the review? Ordering by score alone answers "which is most likely fraud" but treats a 90,000-unit fraud and a 90-million-unit fraud as equivalent - a mistake the cost model explicitly rejects, since a missed fraud costs the full amount lost ($c_{FN} = 1.0\times$ amount).

Ranking instead by *expected loss* - risk score × amount - makes the queue order literally the expected value of the loss each flag represents, aligning the scarce review resource with the very cost function it exists to minimise. The data make this lever unusually powerful because risk and size compound: the top amount decile carries about a 0.69% fraud rate (roughly 30× the low deciles) *and* the largest per-case exposure, so expected loss concentrates on the same transactions from both directions. With total fraud exposure of 12.06 billion units spread across only 8,213 events, a small fraction of large-amount cases dominates recoverable loss. This is where the analytics create the most business value: the model identifies what is *risky*, but expected-loss prioritisation decides what is *worth a human's finite time*, maximising loss avoided per analyst-hour rather than mere hit-count.

9.2 Limitations

- PaySim is a simulation; its post-transaction balance dynamics are cleaner than reality - precisely why the leaky/deployable split (Chapters 5–6) matters and why we never ship balance features.
- The contextual layer is synthetic; its parameters are plausible but not calibrated to a real platform, and real fraud would show richer cross-signal interactions.
- The cost coefficients are business assumptions; only their *ratio* matters (Section 6.4), and a sensitivity sweep is still recommended.
- Operating precision is low *by design*; in production it is bounded by review capacity, an operational constraint that should be modelled explicitly.
- In-app retraining adapts on freshly generated, current-distribution data for the live demo; a production retrain would instead rely on real labelled feedback, which arrives with delay (chargeback / confirmation lag) and needs its own validation.

The operating precision of 0.173 should not be mistaken for a modelling weakness. With a 774:1 base rate and a break-even at 25 currency units - trivially below the median fraud of tens of thousands - even a Bayes-optimal detector is compelled to cast a wide net, because blocking is simply cheaper than the loss it prevents. Precision here is thus an operational dial set by how much

review capacity the business funds, not a fixed property of the classifier, and the sharper question is not "how do we raise precision" but "how many analyst-hours are worth buying" - which the expected-loss ranking answers directly by exposing the diminishing recoverable exposure as the queue is worked downward. Stating precision alongside that capacity constraint, rather than reporting it as if it were free to improve, is the honest way to frame the fraud/friction trade-off.

9.3 Future work

- Publish the live link and add probability calibration (isotonic/Platt) so scores are interpretable as probabilities across models.
- A cost-coefficient sensitivity sweep and a capacity-constrained operating point.
- A full-data ensemble build plus explicit imbalance experiments (SMOTE vs. undersampling vs. class weights).
- Graph features on the destination network for stronger mule detection, and a learned/weighted ensemble instead of the current max-risk union.
- Rolling-window performance monitoring (with label delay) and an automatically triggered - not just dashboard-initiated - retraining pipeline.

9.4 Presentation and demo

The deliverable is presented to the class through the live eight-page Streamlit application: the narrative runs pitch → problem (Segment Analytics) → product (Review Queue) → rigour (Model Evaluation) → value (Cost & ROI) → real-time (Live Feed) → operations (Monitoring) → serving (API Tester), mirroring the structure of this report.

Chapter 10

Conclusion

We built a reproducible, end-to-end fraud-detection product on PaySim augmented with a documented synthetic risk layer, spanning the full eight-module lifecycle from raw data to a monitored deployment. The project's distinctive contribution is methodological rigour under a realistic constraint: by separating leaky post-transaction signals from deployable authorization-time signals, we avoid the near-perfect-but-useless result that balance features invite, and instead deliver a model the business can actually run. The reference XGBoost model reaches a test AUC-PR of 0.907, recall of 98.1%, and avoids 99.9% of fraud loss while flagging only 2.4% of transactions - an operating point chosen not by accuracy but by an explicit business cost function.

That model is productionised as a three-model max-risk ensemble behind a real-time scoring API and an eight-page analyst application, deployable to a single free-tier port, and kept honest by a PSI-based monitoring layer with a retraining trigger, in-app retraining, and webhook alerting. The result is not just a classifier but a complete, auditable analytics product: it scores transactions, explains its flags with reason codes, prices its decisions in currency, and watches itself for drift - closing the loop the brief asks for. The one remaining task is to publish the public live link and attach demo screenshots.

Stepping back from the pipeline, the data taught us a coherent portrait of this fraud, and each finding forced a specific design choice rather than a generic one. Fraud is *rare* (0.129%, a 774:1 imbalance) - so the model is tuned against a business cost function and judged by AUC-PR, not the accuracy the majority class would trivially win. It is *nocturnal* (0.10% by day rising to 1.77% at night, peaking near 22% in the 04:00–05:00 window) and *large-amount* (a 0.69% rate in the top decile, about 30× the bottom) - so time-of-day and log-scaled amount became core features and the review queue is ranked by expected loss. It flows exclusively through `TRANSFER` and `CASH_OUT`, the two exfiltration channels, with literally zero fraud in `PAYMENT`, `CASH_IN`, or `DEBIT` - so a cheap structural pre-filter removes most of the stream before scoring. It is *channelled into reused mule accounts*, the destination fraud rate climbing monotonically from 0.118% at first use to 1.692% beyond eleven reuses - so destination-reuse history became a feature and graph structure is the clearest avenue for future gains. And it is *non-linear*: no signal correlates with the target above $|r| = 0.08$, yet the tree ensemble reaches 0.907 AUC-PR while logistic regression manages only 0.587 - so the deployed model is a gradient-boosted tree, chosen because the phenomenon lives in the interactions, not the margins.

The one-sentence lesson. Fraud in this data is rare, nocturnal, large, funnelled through `TRANSFER`/`CASH_OUT` into reused mule accounts, and invisible to any single signal - so the winning system is not the most *accurate* classifier but the one that integrates weak, overlapping evidence into a calibrated score, respects authorization-time causality, and spends its scarce human review where *expected loss* is largest.

Appendix A

Appendix

A.1 Full model comparison

Table A.1 lists all fifteen (model × feature-group) configurations. Leaky groups (`base`, `all`) are shaded conceptually as upper-bound references.

Table A.1: Full results - all models × feature groups (test metrics; threshold selected on validation).

Model	Group	Leaky	#f	val AUC-PR	test AUC-PR	Prec.	Recall	F1	Loss avd.
Logistic Reg.	base	yes	18	0.523	0.774	0.119	0.992	0.213	98.85%
Logistic Reg.	dest	no	16	0.001	0.011	0.004	0.998	0.008	99.94%
Logistic Reg.	synth	no	21	0.065	0.217	0.010	1.000	0.020	100.0%
Logistic Reg.	realistic	no	44	0.249	0.587	0.025	0.997	0.049	99.98%
Logistic Reg.	all	yes	50	0.604	0.853	0.114	0.989	0.205	98.85%
Random Forest	base	yes	18	1.000	1.000	1.000	0.9995	0.9998	99.99%
Random Forest	dest	no	16	0.001	0.006	0.006	0.803	0.012	85.54%
Random Forest	synth	no	21	0.289	0.138	0.030	0.751	0.058	75.55%
Random Forest	realistic	no	44	0.744	0.812	0.084	0.953	0.155	99.60%
Random Forest	all	yes	50	1.000	1.000	1.000	0.9995	0.9998	99.99%
XGBoost	base	yes	18	1.000	1.000	1.000	0.9995	0.9998	99.99%
XGBoost	dest	no	16	0.001	0.009	0.004	0.998	0.008	99.77%
XGBoost	synth	no	21	0.268	0.243	0.011	0.961	0.022	96.21%
XGBoost	**realistic**	**no**	**44**	**0.806**	**0.907**	**0.173**	**0.981**	**0.294**	**99.91%**
XGBoost	all	yes	50	1.000	1.000	1.000	0.9995	0.9998	99.99%

A.2 Reproducibility

- Fixed seed `SEED=42` throughout; deterministic synthetic generation.
- Full rebuild: `./scripts/get_data.sh` (Kaggle download) then `./train.sh full` runs Modules 1–5 on all 6.36M rows.
- Per-module entry points are listed in Table 1.3.
- Environment pinned in `requirements.txt` (pandas, scikit-learn, xgboost, imbalanced-learn, fastapi, streamlit, evidently).

Bibliography

[1] E. A. Lopez-Rojas, A. Elmir, and S. Axelsson. *PaySim: A financial mobile money simulator for fraud detection.* 28th European Modeling and Simulation Symposium (EMSS), 2016.

[2] R. Roy. *Online Payments Fraud Detection Dataset.* Kaggle. https://www.kaggle.com/datasets/rupakroy/online-payments-fraud-detection-dataset

[3] T. Chen and C. Guestrin. *XGBoost: A Scalable Tree Boosting System.* KDD, 2016.

[4] N. V. Chawla et al. *SMOTE: Synthetic Minority Over-sampling Technique.* JAIR, 2002.

[5] Evidently AI. *Open-source ML monitoring.* https://www.evidentlyai.com/